

10 Practice PySpark DF Operations -NikhilSharma

PySpark DataFrame Operations

This document serves as a comprehensive guide to performing various data manipulation tasks using PySpark DataFrames. We will cover the essential steps of setting up a SparkSession, creating DataFrames with explicit schemas (DDL), and then tackling a series of common analytical queries.

1. Setting up the SparkSession

The `SparkSession` is the entry point to programming Spark with the `DataFrame` and `Dataset` API. It allows you to interact with Spark's core functionalities.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql import Window
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DateType, DoubleType

# Create a SparkSession
# .appName("PySparkDataFrameOperations") sets the name of your Spark application.
# .config("spark.some.config.option", "some-value") allows you to set Spark configurations.
# .getOrCreate() retrieves an existing SparkSession or creates a new one if none exists.
spark = SparkSession.builder \
    .appName("PySparkDataFrameOperations") \
    .getOrCreate()

print("SparkSession created successfully!")
```

2. Creating DataFrames with DDL Schema

Defining a schema (DDL - Data Definition Language) explicitly for your DataFrames is a good practice. It ensures data types are correctly interpreted, which can prevent errors and optimize performance.

2.1. df_dept DataFrame

This DataFrame holds information about departments.

Schema Definition:

```
dept_schema = StructType([
    StructField("department_id", IntegerType(), True),
    StructField("department_name", StringType(), True),
    StructField("location_id", IntegerType(), True)
])
```

Data:

```
dept_data = [
    (50, "Marketing", 1800),
    (60, "Purchasing", 1700),
    (80, "Human Resources", 2400),
    (100, "Shipping", 1500),
    (110, "IT", 1400),
    (120, "Public Relations", 2700),
    (130, "Sales", 2500),
    (140, "Executive", 1700),
    (150, "Finance", 1700),
    (160, "Accounting", 1700),
    (170, "Treasury", 1700),
    (180, "Corporate Tax", 1700),
    (190, "Control And Credit", 1700),
    (200, "Shareholder Services", 1700),
    (210, "Benefits", 1700),
    (220, "Payroll", 1700)
]
```

DataFrame Creation:

```
df_dept = spark.createDataFrame(data=dept_data, schema=dept_schema)
print("df_dept created:")
df_dept.show()
df_dept.printSchema()
```

Output:

```
df_dept created:
+-----+-----+-----+
|department_id|department_name|location_id|
+-----+-----+-----+
|50|Marketing|1800|
|60|Purchasing|1700|
|80|Human Resources|2400|
```

	100	Shipping	1500
	110	IT	1400
	120	Public Relations	2700
	130	Sales	2500
	140	Executive	1700
	150	Finance	1700
	160	Accounting	1700
	170	Treasury	1700
	180	Corporate Tax	1700
	190	Control And Credit	1700
	200	Shareholder Services	1700
	210	Benefits	1700
	220	Payroll	1700

```

root
├── department_id: integer (nullable = true)
├── department_name: string (nullable = true)
└── location_id: integer (nullable = true)

```

2.2. df_employees DataFrame

This DataFrame stores detailed employee information.

Schema Definition:

Note: hire_date is initially read as StringType and then cast to DateType for date operations. commission_pct and manager_id are nullable.

```

employees_schema = StructType([
    StructField("employee_id", IntegerType(), True),
    StructField("first_name", StringType(), True),
    StructField("last_name", StringType(), True),
    StructField("email", StringType(), True),
    StructField("phone_number", StringType(), True),
    StructField("hire_date_str", StringType(), True), # Temporary string field for date
    StructField("job_id", StringType(), True),
    StructField("salary", IntegerType(), True),
    StructField("commission_pct", DoubleType(), True),
    StructField("manager_id", IntegerType(), True),
    StructField("department_id", IntegerType(), True)
])

```

Data:

```

employees_data = [
    (100, "Leonardo", "DiCaprio", "LDICAPRIO", "515.123.4567", "1987-06-17", "AD_PRES", 24000, None, None, 90),
    (101, "Scarlett", "Johansson", "SJOHANSS", "515.123.4568", "1989-11-21", "AD_VP", 17000, None, 100, 90),
    (102, "Robert", "Downey", "RDOWNEY", "515.123.4569", "1993-09-12", "AD_VP", 17000, None, 100, 90),
    (103, "Shah Rukh", "Khan", "SRKHAN", "590.423.4567", "1990-09-30", "IT_PROG", 9000, None, 102, 60),
    (104, "Aamir", "Khan", "AKHAN", "590.423.4568", "1991-05-21", "IT_PROG", 6000, None, 103, 60),
    (105, "Salman", "Khan", "SAKHAN", "590.423.4569", "1997-06-25", "IT_PROG", 4800, None, 103, 60),
    (106, "Priyanka", "Chopra", "PCHOPRA", "590.423.4560", "1998-02-05", "IT_PROG", 4800, None, 103, 60),
    (107, "Deepika", "Padukone", "DPADUKONE", "590.423.5567", "1999-02-09", "IT_PROG", 4200, None, 103, 60),
    (108, "Meryl", "Streep", "MSTREEP", "515.124.4569", "1994-08-17", "FI_MGR", 12000, None, 101, 100),
    (109, "Tom", "Hanks", "THANKS", "515.124.4169", "1994-08-12", "FI_ACCOUNT", 9000, None, 108, 100),
    (110, "Denzel", "Washington", "DWASHING", "515.124.4269", "1997-04-09", "FI_ACCOUNT", 8200, None, 108, 100),
    (111, "Amitabh", "Bachchan", "ABACHCHAN", "515.124.4369", "1997-02-01", "FI_ACCOUNT", 7700, None, 108, 100),
    (112, "Akshay", "Kumar", "AKUMAR", "515.124.4469", "1998-06-03", "FI_ACCOUNT", 7800, None, 108, 100),
    (113, "Hrithik", "Roshan", "HROSHAN", "515.124.4567", "1999-12-07", "FI_ACCOUNT", 6900, None, 108, 100),
    (114, "Will", "Smith", "WSMITH", "515.127.4561", "1994-11-08", "PU_MAN", 11000, None, 100, 30),
    (115, "Ranveer", "Singh", "RSINGH", "515.127.4562", "1995-05-12", "PU_CLERK", 3100, None, 114, 30),
    (116, "Alia", "Bhatt", "ABHATT", "515.127.4563", "1997-12-13", "PU_CLERK", 2900, None, 114, 30),
    (117, "Katrina", "Kaif", "KKAIF", "515.127.4564", "1997-09-10", "PU_CLERK", 2800, None, 114, 30),
    (118, "Ryan", "Reynolds", "RREYNOLDS", "515.127.4565", "1998-01-02", "PU_CLERK", 2600, None, 114, 30),
    (119, "Emma", "Stone", "ESTONE", "515.127.4566", "1999-04-08", "PU_CLERK", 2500, None, 114, 30),
    (120, "Brad", "Pitt", "BPITT", "650.123.1234", "1996-07-18", "ST_MAN", 8000, None, 100, 50),
    (121, "Angelina", "Jolie", "AJOLIE", "650.123.2234", "1997-08-09", "ST_MAN", 8200, None, 100, 50),
    (122, "Ranbir", "Kapoor", "RKAPOOR", "650.123.3234", "1995-05-01", "ST_MAN", 7900, None, 100, 50),
    (123, "Kareena", "Kapoor", "KKAPOOR", "650.123.4234", "1997-10-12", "ST_MAN", 6500, None, 100, 50),
    (124, "Varun", "Dhawan", "VDHAWAN", "650.123.5234", "1999-11-12", "ST_MAN", 5800, None, 100, 50),
    (125, "Jennifer", "Lawrence", "JLAWRENCE", "650.124.1214", "1997-07-02", "ST_CLERK", 3200, None, 120, 50),
    (126, "Anushka", "Sharma", "ASHARMA", "650.124.1224", "1998-11-12", "ST_CLERK", 2700, None, 120, 50),
    (127, "Chris", "Evans", "CEVANS", "650.124.1334", "1999-01-02", "ST_CLERK", 2400, None, 120, 50),
    (128, "Chris", "Hemsworth", "CHEMSW", "650.124.1434", "2000-03-04", "ST_CLERK", 2200, None, 120, 50),
    (129, "Sonam", "Kapoor", "SKAPOOR", "650.124.5234", "1997-09-10", "ST_CLERK", 3300, None, 121, 80),
    (130, "Shahid", "Kapoor", "SHKAPOOR", "650.124.6234", "1997-10-12", "ST_CLERK", 2800, None, 121, 110)
]

```

DataFrame Creation and Date Casting:

```
# Create the DataFrame with the initial schema
df_employees = spark.createDataFrame(data=employees_data, schema=employees_schema)

# Cast the 'hire_date_str' column to DateType and rename it to 'hire_date'
df_employees = df_employees.withColumn("hire_date", F.to_date(F.col("hire_date_str"), "yyyy-MM-dd")) \
    .drop("hire_date_str") # Drop the temporary string column

print("\ndf_employees created:")
df_employees.show(5) # Show first 5 rows
df_employees.printSchema()
```

Output:

```
df_employees created:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|employee_id|first_name|last_name|email|phone_number|job_id|salary|commission_pct|manager_id|department_id|hire_date|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|100|Leonardo|DiCaprio|LDICAPRIO|515.123.4567|AD_PRES|24000|NULL|NULL|90|1987-06-17|
|101|Scarlett|Johansson|SJOHANSS|515.123.4568|AD_VP|17000|NULL|100|90|1989-11-21|
|102|Robert|Downey|RDOWNEY|515.123.4569|AD_VP|17000|NULL|100|90|1993-09-12|
|103|Shah Rukh|Khan|SRKHAN|590.423.4567|IT_PROG|9000|NULL|102|60|1990-09-30|
|104|Aamir|Khan|AKHAN|590.423.4568|IT_PROG|6000|NULL|103|60|1991-05-21|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

only showing top 5 rows
```

```
root
├─ employee_id: integer (nullable = true)
├─ first_name: string (nullable = true)
├─ last_name: string (nullable = true)
├─ email: string (nullable = true)
├─ phone_number: string (nullable = true)
├─ job_id: string (nullable = true)
├─ salary: integer (nullable = true)
├─ commission_pct: double (nullable = true)
├─ manager_id: integer (nullable = true)
├─ department_id: integer (nullable = true)
├─ hire_date: date (nullable = true)
```

3. PySpark Dataframe Operations: Questions and Answers

Now, let's explore various data manipulation techniques using PySpark DataFrames to answer specific business questions.

Question 1: Select employee's first name, last name, job_id, and salary whose first name starts with alphabet 'S'

Code:

```
print("\n--- Question 1: Employees whose first name starts with 'S' ---")
result_q1 = (
    df_employees.filter(F.col('first_name').like('S%'))
    .select('first_name', 'last_name', 'job_id', 'salary')
)
result_q1.show()
```

Output:

```
--- Question 1: Employees whose first name starts with 'S' ---
+-----+-----+-----+-----+
|first_name|last_name|job_id|salary|
+-----+-----+-----+-----+
|Scarlett|Johansson|AD_VP|17000|
|Shah Rukh|Khan|IT_PROG|9000|
|Salman|Khan|IT_PROG|4800|
|Sonam|Kapoor|ST_CLERK|3300|
|Shahid|Kapoor|ST_CLERK|2800|
+-----+-----+-----+-----+
```

Question 2: Find the employees who joined the company after the 15th of the month

Code:

```
print("\n--- Question 2: Employees hired after 15th of the month ---")
result_q2 = (
    df_employees.filter(F.dayofmonth('hire_date') >= 15)
```

```
.select('first_name', 'last_name', 'hire_date')
)
result_q2.show()
```

Output:

```
--- Question 2: Employees hired after 15th of the month ---
+-----+-----+
|first_name|last_name| hire_date|
+-----+-----+
| Leonardo| DiCaprio|1987-06-17|
| Scarlett|Johansson|1989-11-21|
| Shah Rukh| Khan|1990-09-30|
| Aamir| Khan|1991-05-21|
| Salman| Khan|1997-06-25|
| Meryl| Streep|1994-08-17|
| Brad| Pitt|1996-07-18|
+-----+-----+
```

Question 3: Display the employee's first name and first name in reverse order

Code:

```
print("\n--- Question 3: First name and reversed first name ---")
result_q3 = (
    df_employees.select('first_name', F.reverse('first_name').alias('name_reversed'))
)
result_q3.show(10) # Showing top 10 rows as per original
```

Output:

```
--- Question 3: First name and reversed first name ---
+-----+-----+
|first_name|name_reversed|
+-----+-----+
| Leonardo| odranoel|
| Scarlett| ttelracS|
| Robert| treboR|
| Shah Rukh| hkuR hahS|
| Aamir| rimaA|
| Salman| namlaS|
| Priyanka| aknayirP|
| Deepika| akipeeD|
| Meryl| lyreM|
| Tom| moT|
+-----+-----+
only showing top 10 rows
```

Question 4: Display the 5 least earning employees

Code:

```
print("\n--- Question 4: 5 least earning employees ---")
result_q4 = (
    df_employees.select('first_name', 'last_name', 'salary')
    .orderBy(F.asc('salary'))
    .limit(5)
)
result_q4.show()
```

Output:

```
--- Question 4: 5 least earning employees ---
+-----+-----+-----+
|first_name|last_name|salary|
+-----+-----+-----+
| Chris|Hensworth| 2200|
| Chris| Evans| 2400|
| Emma| Stone| 2500|
| Ryan| Reynolds| 2600|
| Anushka| Sharma| 2700|
+-----+-----+-----+
```

Question 5: Find the employees hired in the 80s**Code:**

```
print("\n--- Question 5: Employees hired in the 80s ---")
result_q5 = (
    df_employees.select('first_name', 'last_name', 'hire_date')
    .filter(F.year('hire_date').between(1980, 1989))
)
result_q5.show()
```

Output:

```
--- Question 5: Employees hired in the 80s ---
+-----+-----+-----+
|first_name|last_name| hire_date|
+-----+-----+-----+
| Leonardo| DiCaprio|1987-06-17|
| Scarlett|Johansson|1989-11-21|
+-----+-----+-----+
```

Question 6: Find the maximum salary from each department**Code:**

```
print("\n--- Question 6: Maximum salary per department ---")
joined_df_q6 = df_employees.join(df_dept, df_employees.department_id == df_dept.department_id, 'left')
result_q6 = (
    joined_df_q6.groupBy('department_name')
    .agg(F.max('salary').alias('dept_max_salary'))
)
result_q6.show()
```

Output:

```
--- Question 6: Maximum salary per department ---
+-----+-----+
|department_name|dept_max_salary|
+-----+-----+
| NULL|24000|
| Purchasing|9000|
| Marketing|8200|
| IT|2800|
| Shipping|12000|
| Human Resources|3300|
+-----+-----+
```

_Note: The NULL value under department_name corresponds to employees who are not assigned to any department in the df_dept (dept 90).

Question 7: Find employees who earn more than the average salary in that company**Code:**

```
print("\n--- Question 7: Employees earning more than average salary ---")
average_salary = df_employees.select(F.avg('salary')).first()[0]
print(f"Average salary in the company: {average_salary:.2f}")

result_q7 = (
    df_employees.filter(F.col('salary') > average_salary)
    .select('first_name', 'last_name', 'salary')
)
result_q7.show()
```

Output:

```
--- Question 7: Employees earning more than average salary ---
Average salary in the company: 6977.42
+-----+-----+-----+
|first_name| last_name|salary|
+-----+-----+-----+
| Leonardo| DiCaprio| 24000|
| Scarlett| Johansson| 17000|
| Robert| Downey| 17000|
| Shah Rukh| Khan| 9000|
| Meryl| Streep| 12000|
| Tom| Hanks| 9000|
+-----+-----+-----+
```

	Denzel	Washington		8200	
	Amitabh	Bachchan		7700	
	Akshay	Kumar		7800	
	Will	Smith		11000	
	Brad	Pitt		8000	
	Angelina	Jolie		8200	
	Ranbir	Kapoor		7900	
+	+	+	+	+	+

Question 8: Find the employees who joined in August, 1994**Code:**

```
print("\n--- Question 8: Employees hired in August 1994 ---")
result_q8 = (
    df_employees.filter((F.year('hire_date') == 1994) & (F.month('hire_date') == 8))
    .select(F.concat('first_name', F.lit(' '), 'last_name').alias('emp_name'),
           'hire_date')
)
result_q8.show()
```

Output:

```
--- Question 8: Employees hired in August 1994 ---
+-----+-----+
| emp_name | hire_date |
+-----+-----+
|Meryl Streep|1994-08-17|
| Tom Hanks|1994-08-12|
+-----+-----+
```

Question 9: Select employee first name and the corresponding phone number in the format (_)-(_)-(_ _ _)**Code:**

```
print("\n--- Question 9: Employee name and formatted phone number ---")
result_q9 = (
    df_employees.select(F.concat(F.col('first_name'), F.lit(' '), F.col('last_name')).alias('emp_name'),
                          F.regexp_replace('phone_number', '\\.', '-').alias('phn_number'))
)
result_q9.show(10) # Showing top 10 rows as per original
```

Output:

```
--- Question 9: Employee name and formatted phone number ---
+-----+-----+
| emp_name | phn_number |
+-----+-----+
| Leonardo DiCaprio|515-123-4567|
| Scarlett Johansson|515-123-4568|
| Robert Downey|515-123-4569|
| Shah Rukh Khan|590-423-4567|
| Aamir Khan|590-423-4568|
| Salman Khan|590-423-4569|
| Priyanka Chopra|590-423-4560|
| Deepika Padukone|590-423-5567|
| Meryl Streep|515-124-4569|
| Tom Hanks|515-124-4169|
+-----+-----+
only showing top 10 rows
```

Question 10: Select the employees whose first_name contains "an"**Code:**

```
print("\n--- Question 10: Employees whose first name contains 'an' ---")
result_q10 = (
    df_employees.filter(F.col('first_name').like('%an%'))
    .select('first_name', 'last_name')
)
result_q10.show()
```

Output:

```
--- Question 10: Employees whose first name contains 'an' ---
+-----+-----+
|first_name|last_name|
+-----+-----+
```

Salman	Khan
Priyanka	Chopra
Ranveer	Singh
Ryan	Reynolds
Ranbir	Kapoor

Question 11: Write a query to divide people into three groups based on their salaries

Code:

```
print("\n--- Question 11: Employees divided into salary groups ---")
# Define the salary thresholds for the three groups
thresholds = [2000, 5000, 10000]

# Create a User-Defined Function (UDF) to categorize salaries into groups
from pyspark.sql.types import StringType # Ensure StringType is imported if not already

def categorize_salary(salary):
    if salary is None:
        return None
    if salary >= thresholds[0] and salary < thresholds[1]:
        return 'low'
    elif salary >= thresholds[1] and salary < thresholds[2]:
        return 'mid'
    else:
        return 'high'

# Register the UDF
categorize_salary_udf = F.udf(categorize_salary, StringType())

# Add a new column to the DataFrame indicating the salary group
df_employees_with_salary_group = df_employees.withColumn("salary_group",
                                                         categorize_salary_udf(F.col('salary')))

# Show the DataFrame with the new column
result_q11 = df_employees_with_salary_group.select('first_name', 'last_name', 'salary', 'salary_group').orderBy('salary')
result_q11.show(20)
```

Output:

```
--- Question 11: Employees divided into salary groups ---
+-----+-----+-----+-----+
|first_name|last_name|salary|salary_group|
+-----+-----+-----+-----+
| Chris|Hemsworth| 2200| low|
| Chris| Evans| 2400| low|
| Emma| Stone| 2500| low|
| Ryan| Reynolds| 2600| low|
| Anushka| Sharma| 2700| low|
| Katrina| Kaif| 2800| low|
| Shahid| Kapoor| 2800| low|
| Alia| Bhatt| 2900| low|
| Ranveer| Singh| 3100| low|
| Jennifer| Lawrence| 3200| low|
| Sonam| Kapoor| 3300| low|
| Deepika| Padukone| 4200| low|
| Salman| Khan| 4800| low|
| Priyanka| Chopra| 4800| low|
| Varun| Dhawan| 5800| mid|
| Aamir| Khan| 6000| mid|
| Kareena| Kapoor| 6500| mid|
| Hrithik| Roshan| 6900| mid|
| Amitabh| Bachchan| 7700| mid|
| Akshay| Kumar| 7800| mid|
+-----+-----+-----+-----+
only showing top 20 rows
```

Question 12: Find the salary range of employees (max, min, and average salary)

Code:

```
print("\n--- Question 12: Salary range of employees ---")
result_q12 = (
    df_employees.select(F.max('salary').alias('max_salary'),
```

```

        F.min('salary').alias('min_salary'),
        F.round(F.avg('salary'), 2).alias('avg_salary')) # Round average to 2 decimal places
    )
    result_q12.show()

```

Output:

```

--- Question 12: Salary range of employees ---
+-----+-----+-----+
|max_salary|min_salary|avg_salary|
+-----+-----+-----+
|      24000|      2200|   6977.42|
+-----+-----+-----+

```

Question 13: Get the count of employees hired year wise

Code:

```

print("\n--- Question 13: Employee count hired year-wise ---")
result_q13 = (
    df_employees.groupBy(F.year('hire_date').alias('hire_year'))
    .agg(F.count('employee_id').alias('employee_count'))
    .orderBy('hire_year')
)
result_q13.show()

```

Output:

```

--- Question 13: Employee count hired year-wise ---
+-----+-----+
|hire_year|employee_count|
+-----+-----+
|    1987|             1|
|    1989|             1|
|    1990|             1|
|    1991|             1|
|    1993|             1|
|    1994|             3|
|    1995|             2|
|    1996|             1|
|    1997|            10|
|    1998|             4|
|    1999|             5|
|    2000|             1|
+-----+-----+

```

Question 14: Find the count of employees in each department

Code:

```

print("\n--- Question 14: Employee count per department ---")
# Perform a left join to include departments even if they have no employees
joined_df_q14 = df_dept.join(df_employees, df_employees.department_id == df_dept.department_id, 'left')

result_q14 = (
    joined_df_q14.groupBy(df_dept['department_name'])
    .agg(F.count(df_employees['employee_id']).alias('emp_count')) # Count non-null employee_ids
    .orderBy(F.desc('emp_count'))
)
result_q14.show()

```

Output:

```

--- Question 14: Employee count per department ---
+-----+-----+
|department_name|emp_count|
+-----+-----+
|      Marketing|         9|
|      Shipping|         6|
|    Purchasing|         5|
|           IT|         1|
|Human Resources|         1|
|Corporate Tax|         0|
|         Sales|         0|
|    Accounting|         0|
+-----+-----+

```



```

|          Payroll|          0|
|          Finance|          0|
|   Public Relations|          0|
|          Executive|          0|
|          Treasury|          0|
|Shareholder Services|          0|
|   Control And Credit|          0|
|          Benefits|          0|
+-----+-----+

```

Question 15: Find count of employees under each manager in descending order
Code:

```

print("\n--- Question 15: Employee count under each manager ---")
# Alias DataFrames for self-join
emp_df = df_employees.alias('emp_df')
mgr_df = df_employees.alias('mgr_df')

# Join employees to managers on manager_id = employee_id
joined_df_q15 = emp_df.join(mgr_df, F.col('emp_df.manager_id') == F.col('mgr_df.employee_id'), 'inner')

result_q15 = (
    joined_df_q15.groupBy(F.concat('mgr_df.first_name', F.lit(' '), 'mgr_df.last_name').alias('manager'))
    .agg(F.count(F.col('emp_df.employee_id')).alias('reportee_count'))
    .orderBy(F.desc('reportee_count'))
)
result_q15.show()

```

Output:

```

--- Question 15: Employee count under each manager ---
+-----+-----+
|          manager|reportee_count|
+-----+-----+
| Leonardo DiCaprio|            8|
|      Meryl Streep|            5|
|      Will Smith|            5|
|      Brad Pitt|            4|
|   Shah Rukh Khan|            4|
|   Angelina Jolie|            2|
|Scarlett Johansson|            1|
|   Robert Downey|            1|
+-----+-----+

```

Question 16: Display employees and their corresponding managers and with their salaries

Code:

```

print("\n--- Question 16: Employees, managers, and their salaries ---")
# Alias DataFrames for self-join
emp_df = df_employees.alias('emp_df')
mgr_df = df_employees.alias('mgr_df')

# Join employees to managers on manager_id = employee_id
joined_df_q16 = emp_df.join(mgr_df, F.col('emp_df.manager_id') == F.col('mgr_df.employee_id'), 'inner')

result_q16 = (
    joined_df_q16.select(F.concat('emp_df.first_name', F.lit(' '), 'emp_df.last_name').alias('emp_name'),
                          F.col('emp_df.salary').alias('emp_salary'),
                          F.concat('mgr_df.first_name', F.lit(' '), 'mgr_df.last_name').alias('manager_name'),
                          F.col('mgr_df.salary').alias('manager_salary'))
)
result_q16.show(10) # Showing top 10 rows as per original

```

Output:

```

--- Question 16: Employees, managers, and their salaries ---
+-----+-----+-----+-----+
| emp_name|emp_salary| manager_name|manager_salary|
+-----+-----+-----+-----+
|Scarlett Johansson|    17000| Leonardo DiCaprio|    24000|
|   Robert Downey|    17000| Leonardo DiCaprio|    24000|
|      Will Smith|    11000| Leonardo DiCaprio|    24000|
|      Brad Pitt|     8000| Leonardo DiCaprio|    24000|

```

```
| Angelina Jolie|      8200| Leonardo DiCaprio|      24000|
|  Ranbir Kapoor|      7900| Leonardo DiCaprio|      24000|
|  Kareena Kapoor|      6500| Leonardo DiCaprio|      24000|
|   Varun Dhawan|      5800| Leonardo DiCaprio|      24000|
|   Meryl Streep|     12000| Scarlett Johansson|     17000|
|   Shah Rukh Khan|      9000|   Robert Downey|     17000|
+-----+-----+-----+-----+
only showing top 10 rows
```

Question 17: Display employees first name, last name, job_id and salary whose first name starts with alphabet S

Code:

```
print("\n--- Question 17: Employees whose first name starts with 'S' (Duplicate of Q1) ---")
result_q17 = (
    df_employees.filter(F.col('first_name').like('S%'))
    .select('first_name', 'last_name', 'job_id', 'salary')
)
result_q17.show()
```

Output:

```
--- Question 17: Employees whose first name starts with 'S'
+-----+-----+-----+-----+
|first_name|last_name|  job_id|salary|
+-----+-----+-----+-----+
|  Scarlett|Johansson|  AD_VP| 17000|
|   Shah Rukh|   Khan| IT_PROG|  9000|
|    Salman|   Khan| IT_PROG|  4800|
|    Sonam|  Kapoor|ST_CLERK|  3300|
|   Shahid|  Kapoor|ST_CLERK|  2800|
+-----+-----+-----+-----+
```

Question 18: Find employee with the highest salary

Code:

```
print("\n--- Question 18: Employee with the highest salary ---")
# Define a window specification to order by salary in descending order
# from pyspark.sql import Window # Ensure Window is imported

windowSpec_q18 = Window.orderBy(F.desc('salary'))

result_q18 = (
    df_employees.withColumn('rnk', F.dense_rank().over(windowSpec_q18))
    .filter(F.col('rnk') == 1)
    .select('employee_id', 'first_name', 'last_name', 'salary')
)
result_q18.show()
```

Output:

```
--- Question 18: Employee with the highest salary ---
+-----+-----+-----+-----+
|employee_id|first_name|last_name|salary|
+-----+-----+-----+-----+
|         100|  Leonardo|  DiCaprio| 24000|
+-----+-----+-----+-----+
```

Question 19: Fetch employees with the nth highest salary

Code:

```
print("\n--- Question 19: Employees with the Nth highest salary ---")
# from pyspark.sql import Window # Ensure Window is imported

def get_top_ranked_employees(df, rank):
    windowSpec = Window.orderBy(F.desc('salary'))
    result = (
        df.withColumn('rnk', F.dense_rank().over(windowSpec))
        .filter(F.col('rnk') == rank)
        .select('employee_id', 'first_name', 'last_name', 'salary')
    )
    return result

# Example: Get the second-highest salary employee
```

```
print("\n2nd highest salary:")
rank_2_result = get_top_ranked_employees(df_employees, 2)
rank_2_result.show()

# Example: Get the third-highest salary employee
# print("\n3rd highest salary:")
# rank_3_result = get_top_ranked_employees(df_employees, 3)
# rank_3_result.show()
```

Output (2nd highest salary):

```
--- Question 19: Employees with the Nth highest salary ---
```

```
2nd highest salary:
```

employee_id	first_name	last_name	salary
101	Scarlett	Johansson	17000
102	Robert	Downey	17000

Question 20: Find the managers and the reporting employees who work in different departments

Code:

```
print("\n--- Question 20: Managers and reports in different departments ---")
# Rename columns in the first DataFrame instance to avoid ambiguity
emp_df = (
    df_employees.withColumnRenamed("department_id", "emp_department_id")
    .withColumnRenamed("first_name", "emp_first_name")
    .withColumnRenamed("last_name", "emp_last_name")
    .alias('emp_df')
)

# Rename columns in the second DataFrame instance (for managers)
mgr_df = (
    df_employees.withColumnRenamed("department_id", "mgr_department_id")
    .withColumnRenamed("first_name", "mgr_first_name")
    .withColumnRenamed("last_name", "mgr_last_name")
    .alias('mgr_df')
)

# Join employees to managers on manager_id = employee_id
joined_df_q20 = emp_df.join(mgr_df, F.col('emp_df.manager_id') == F.col('mgr_df.employee_id'), 'inner')

# Filter for employees and managers in different departments
filtered_df_q20 = (
    joined_df_q20.filter(F.col('emp_df.emp_department_id') != F.col('mgr_df.mgr_department_id'))
    .select(F.concat(F.col('mgr_df.mgr_first_name'), F.lit(' '), F.col('mgr_df.mgr_last_name')).alias('manager_name'),
        F.col('mgr_df.mgr_department_id').alias('manager_department_id'),
        F.concat(F.col('emp_df.emp_first_name'), F.lit(' '), F.col('emp_df.emp_last_name')).alias('emp_name'),
        F.col('emp_df.emp_department_id').alias('emp_department_id'))
)
filtered_df_q20.show(10) # Showing top 10 rows
```

Output:

```
--- Question 20: Managers and reports in different departments ---
```

manager_name	manager_department_id	emp_name	emp_department_id
Leonardo DiCaprio	90	Will Smith	30
Leonardo DiCaprio	90	Brad Pitt	50
Leonardo DiCaprio	90	Angelina Jolie	50
Leonardo DiCaprio	90	Ranbir Kapoor	50
Leonardo DiCaprio	90	Kareena Kapoor	50
Leonardo DiCaprio	90	Varun Dhawan	50
Scarlett Johansson	90	Meryl Streep	100
Robert Downey	90	Shah Rukh Khan	60
Angelina Jolie	50	Sonam Kapoor	80
Angelina Jolie	50	Shahid Kapoor	110